

Bases de données avancées

Introduction au langage PL/SQL



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Introduction



Pourquoi PL/SQL

- SQL ne traite qu'un ensemble de données dotées de critère de recherche.
- Les notions de programmation ne sont pas valides dans SQL
- PL/SQL est une extension procédurale de SQL incluant:
 - Les variables et les types
 - Les structures de contrôle et les boucles.
 - Les procédures et les fonctions.
 - Les types d'objets et les méthodes.
- **Objectifs :**
 - cohabiter les instructions procédurales (boucles, conditions...), avec des instructions SQL
 - créer des traitements complexes destinés à être stockés sur le serveur de base de données (objets serveur)



C'est quoi **PL/SQL**?

- **PL/SQL** (Procedural Language / Structured Query Language)
- PL/SQL est une extension procédurale sur Oracle du SQL.
- Permet de combiner les déclarations SQL avec les déclarations procédurales comme IF, structures de boucle, etc.
- PL/SQL utilise SQL pour la manipulation des données et utilise ses propres déclarations pour le traitement.



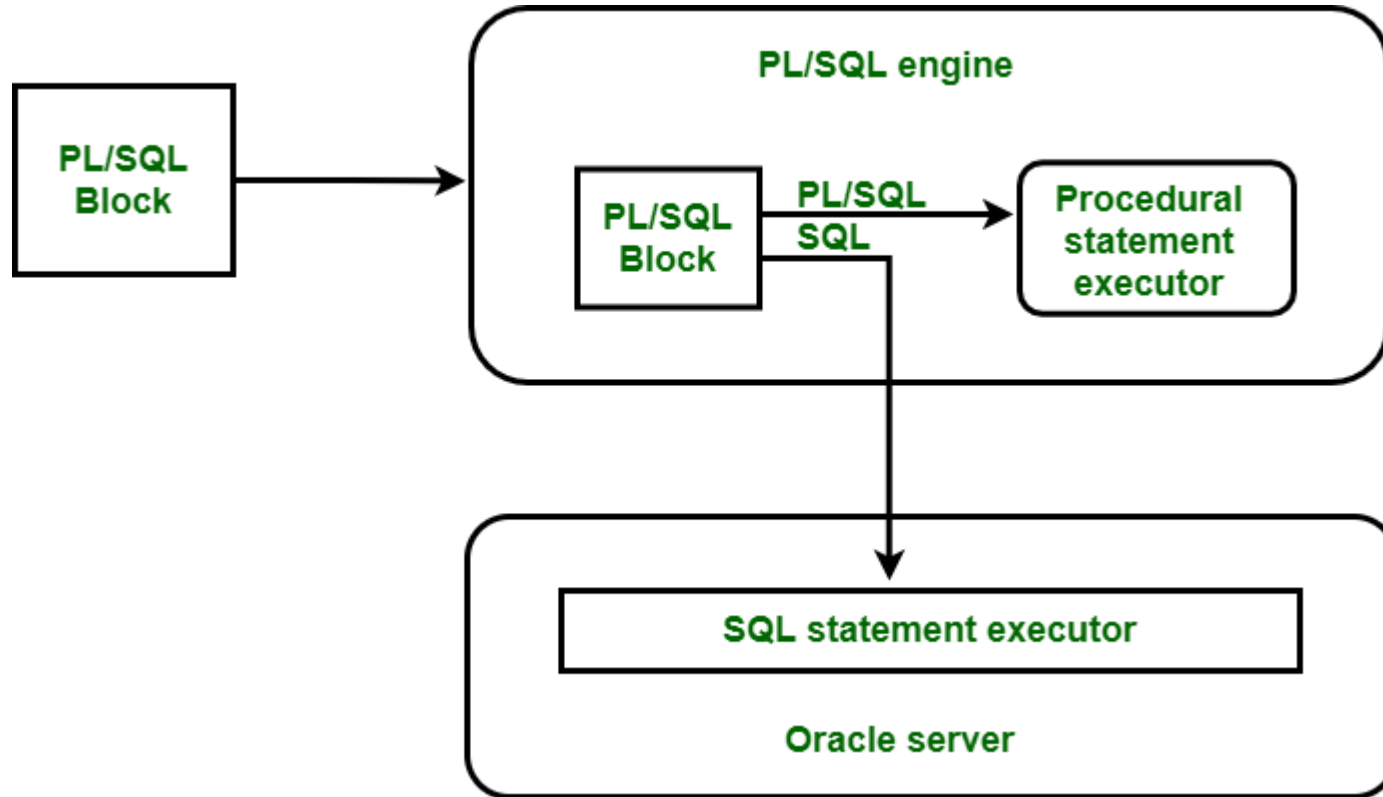
Composants de PL/SQL

Les unités d'un programme PL/SQL sont catégorisées comme suit:

- **blocs anonymes**: apparaissent lorsque des instructions SQL peuvent apparaître.
- **procédures stockées**: elles le sont dans la base de données avec un nom. Des programmes peuvent les exécuter en utilisant ce nom.
- Oracle permet de créer des **fonctions**, similaires aux procédures mais elles retournent une valeur.
- Il permet aussi de créer des **packages**
- Chaque bloc PL/SQL est exécuté par un **moteur** PL/SQL. Ce moteur est chargé de **compiler et exécuter** ces blocs.
- Ce moteur est disponible dans des outils de Oracle Server comme: Oracle Forms et Oracle Reports.
- Durant son exécution, il envoie des commandes SQL à **l'exécuteur de commandes** dans le SGBD Oracle.
 - Càd, il sépare les commandes SQL de celles de PL/SQL.
 - Les commandes PL/SQL sont exécutées par "Procedural Statement Executor"



Composants de PL/SQL





Caractéristiques de PL/SQL -1

Structure de Bloc

- Tout programme PL/SQL est écrit en blocs,
- Les blocs peuvent aussi être imbriqués.
- Chaque bloc est conçu pour une tâche particulière.

Variables et constantes

- PL/SQL permet de déclarer des variables et des constantes.
- Les variables sont utilisées pour stocker des valeurs temporairement.

Structures de contrôle

- PL/SQL permet d'utiliser **IF**, la boucle **FOR**, la boucle **WHILE** dans le bloc. Ces structures constituent une importante extension du SQL au PL/SQL. Elles permettent tout traitement de données en PL/SQL.

Gestion des exceptions

- PL/SQL permet de détecter des erreurs et de les gérer. Chaque fois qu'il y ait une erreur prédéfinie, PL/SQL soulève automatiquement une exception.
- Ces exceptions peuvent être manipulées pour récupérer les erreurs.



Caractéristiques de PL/SQL -2

Modularité

- PL / SQL permet à un processus d'être divisé en différents **modules**.
- Les sous-programmes appelés procédures et fonctions peuvent être définies et appelées à l'aide d'un nom.
- Ces sous-programmes peuvent également prendre des paramètres.

Curseurs

- Un curseur est une zone SQL privée utilisée pour exécuter des instructions SQL et stocker des informations de traitement.
- PL/SQL utilise implicitement les curseurs pour toutes les commandes LMD et la commande SELECT qui retourne une seule ligne.
- Il permet également de définir un curseur explicite pour faire face aux multiples requêtes de ligne.

2

Les bases de PL/SQL



- Les programmes PL/SQL sont écrits sous forme de blocs.
- Un bloc permet de grouper les déclarations liées de manière logique.
- Un bloc est constitué de trois parties suivantes:
 - Partie déclarative
 - Partie exécutable
 - Partie de gestion des exceptions



DECLARE

-- zone de déclaration de variables

BEGIN

-- traitements ...

EXCEPTION

-- traitement des erreurs rencontrées

END;

/

Section optionnelle

Seuls BEGIN et END
sont obligatoires

Section optionnelle :
permet de traiter les
erreurs retournées par
le SGBD à la suite
d'exécutions
d'instructions SQL.

Comme les instruction SQL, les
bloc se terminent par un ;

Un bloc peut être imbriqué dans le code d'un autre bloc



La section déclarative :

- déclaration de tous les types, variables et constantes nécessaires à l'exécution du bloc.
- Ces variables peuvent être de n'importe quel type SQL ou PL/SQL

DECLARE

Nbre NUMBER(3) := 0 ;

Ddate DATE := SYSDATE ;

Cchaine VARCHAR(10) := 'PL/SQL' ;

CPi CONSTANT NUMBER := 3.1415926535 ;



La section exécution :

- Obligatoire et délimitée par les mots clé **BEGIN** et **END**; elle contient:
 - les instructions de contrôle et d'itération.
 - l'appel des procédures et fonctions,
 - l'utilisation des fonctions natives,
 - les ordres SQL, etc.
- Chaque instruction doit être suivie du terminateur d'instruction ;



Bloc imbriqué

- Il est possible de définir un bloc à l'intérieur d'un autre bloc.
- Lorsque les blocs sont définis à l'intérieur de l'un l'autre, ils sont dits **imbriqués**.
- Ci-après un exemple de bloc imbriqué. Le bloc principal contient une seule variable X et le bloc imbriqué contient une seule variable Y;

```
declare
  x number(5);
begin
  -- mettre le code exécutable du bloc principal ici
  declare /* début du bloc imbriqué */
    y number(3);
  begin
    -- mettre le code exécutable du bloc imbriqué
  exception
    -- la gestion des exceptions pour le bloc imbriqué
  end;
  -- code de bloc principal se poursuit.
  Exception
    -- la gestion des exceptions pour le bloc principal
end;/
```



Portée de **variables**

```
DECLARE
    x    INTEGER;
BEGIN
    ...
    DECLARE
        y    NUMBER;
    BEGIN
        ...
    END ;
    ...
END ;
```

Portée de x

Portée de y



Règles syntaxiques

Identifiants

- Peuvent contenir jusqu'à 30 caractères.
- Ne peuvent pas contenir de mots réservés à moins qu'ils soient encadrés de guillemets.
- Doivent commencer par une lettre et peut contenir lettres, chiffres, _, \$ et #
- Pas sensible à la casse
- Doivent avoir un nom distinct de celui d'une table de la base ou d'une colonne.

Autres

- Utiliser un slash (/) pour exécuter un bloc PL/SQL anonyme dans PL/SQL
- Les chaînes de caractères et les dates doivent être entourées de simples quotes (' ').
- Les commentaires peuvent être :
 - sur plusieurs lignes avec : /* ... */
 - sur une ligne précédée de : --



Déclaration et initialisation de **variables** -1

identifieur [CONSTANT] datatype [NOT NULL] [:= | DEFAULT expr];

- Adopter les conventions de dénomination des objets.
- Initialiser les constantes et les variables déclarées NOT NULL.
- Initialiser les identifiants en utilisant l'opérateur d'affectation (:=) ou le mot réservé DEFAULT.
- Déclarer au plus un identifiant par ligne => Déclaration multiple interdite



Types de variables -1

Types scalaires



Variables scalaires : NUMBER(5,2), CHAR, VARCHAR2, DATE, BOOLEAN, ...

Exemple

```
v_gender          CHAR( 1 );  
v_count          BINARY_INTEGER := 0;  
v_total_sal      NUMBER( 9, 2 ) := 0;  
v_order_date     DATE := SYSDATE;  
c_tax_rate       CONSTANT NUMBER ( 3, 2 ) := 8.25;  
v_valid          BOOLEAN NOT NULL := TRUE;
```



Types de **variables** -2

```
c          CHAR( 1 );
name       VARCHAR2(10) := 'Scott';
cpt        BINARY_INTEGER := 0;
total      NUMBER( 9, 2 ) := 0;
order      DATE := SYSDATE + 7;
Ship       DATE;
pi         CONSTANT NUMBER ( 3, 2 ) := 3.14;
done       BOOLEAN NOT NULL := TRUE;
ID         NUMBER(3) NOT NULL := 201;
PRODUIT NUMBER(4) := 2*100;
V_date    DATE := TO_DATE('17-OCT-01', 'DD-MON-YY');
V1        NUMBER := 10;
V2        NUMBER := 20;
V3        BOOLEAN := (v1>v2);
Ok        BOOLEAN := (z IS NOT NULL);
```



Types de variables -3

L'attribut %TYPE

Déclarer une variable à partir :

- D'une autre variable déclarée précédemment
- De la définition d'une colonne de la base de données

Préfixer %TYPE avec :

- La table et la colonne de la base de données
- Le nom de la variable déclarée précédemment
- PL/SQL détermine le type de donnée et la taille de la variable.

L'attribut %ROWTYPE

- Le nombre de colonnes, ainsi que les types de données des colonnes de la table de référence peuvent être inconnus.
- Le nombre de colonnes, ainsi que le type des colonnes de la table de référence peuvent changer en exécution

Utile lorsqu'on recherche

- Une ligne avec l'ordre SELECT.
- Plusieurs lignes avec un curseur explicite.



Types de variables -4

Exemple %TYPE

```
DECLARE
    vnom      emp.nom%TYPE;

BEGIN
    SELECT  nom, sal
    INTO    vnom, vSal
    FROM      emp
    WHERE     numemp= 1000;

END;
```

Exemple %ROWTYPE

```
DECLARE
    emp_record employees%ROWTYPE;
BEGIN
    SELECT * INTO emp_record
    FROM employees
    WHERE id = 101;

    DBMS_OUTPUT.PUT_LINE('Nom: ' ||
emp_record.nom || ', Salaire: ' || emp_record.salaire);
END;
```



Exemple code PL/SQL

-1

```
DECLARE
    vchaine VARCHAR(40) := 'Salut Monde' ;
BEGIN
    DBMS_OUTPUT.PUT_LINE( vchaine ) ;
END ;
/
```

- assignation : en utilisant l'opérateur :=
- Ici, une la variable vchaine est déclarée de type VARCHAR(40) et initialisée avec la valeur 'Salut Monde' ;
- Dans la section exécutable, cette variable est transmise à DBMS_OUTPUT.PUT_LINE pour être affichée à l'écran.
- Ajouter la commande SET SERVEROUTPUT ON; au début pour afficher une sortie en PL/SQL



Exemple code PL/SQL

-2

- L'exemple suivant copie le montant total payé par l'étudiant ayant rollno 102 dans la variable V_SUM, qui est déclarée comme NUMBER (5)

```
declare
  v_sum    number(5);
  v_frais  cours.frais%type;
  v_dur    cours.durée%type;

begin
  select sum(amount) into v_sum from paiement where rollno =
    102;
  -- récupérer les frais et la durée du cours Oracle
  select frais, durée into v_frais, v_duration from cours
    where ccode = 'ora';
  ...
end;
```



Exercice -1

Supposons qu'il y a une table appelée COURS contenant :

- une colonne **nom_cours** contenant des valeurs comme 'C++', 'JAVA'
- une colonne **duree** représentant la durée du cours (en heures, par exemple)

Travail à faire: Écrire un bloc PL/SQL qui change la durée des cours C++ à celle des cours de JAVA

```
BEGIN
  UPDATE cours
  SET duree = (
    SELECT duree
    FROM cours
    WHERE nom_cours = 'JAVA'
  )
  WHERE nom_cours = 'C++';

END;
/
```




Exercice -2

Ecrire un bloc PL/SQL permettant d'interroger une table nommée 'emp' et permettant d'afficher le nom de l'employé numéro 123 et de charger son salaire dans une variable temporaire

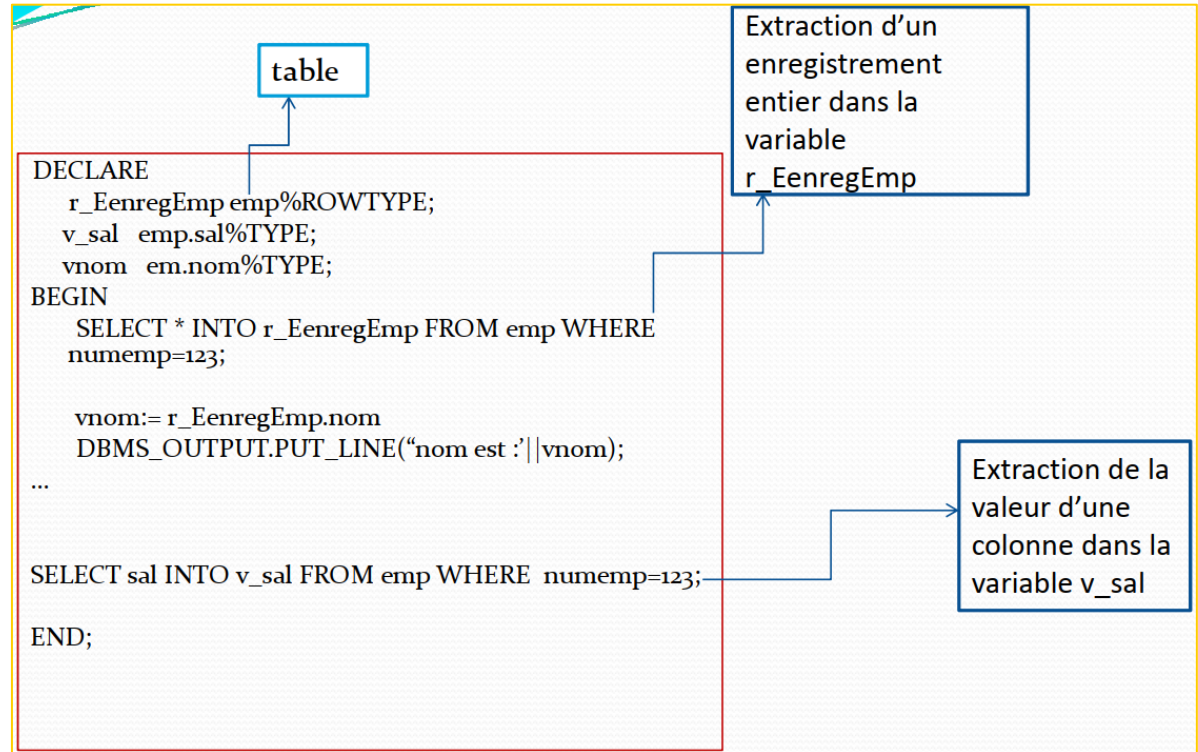




Tableau en PL/SQL

Syntaxe : en deux étapes

1. TYPE nomTypeTableau IS TABLE OF {TypeScalaire|variable%TYPE|table%ROWTYPE|Table.colonne%type};
2. nomTableau nomTypeTableau;

```
DECLARE
    TYPE brv_typeTab is TABLE OF VARCHAR2(6);
    tabBrevet brv_typeTab;

    TYPE nbr_typeTab is TABLE OF NUMBER;
    tabnbr brv_typeTab;
BEGIN
    tabBrevet(1) := 'PL1';
    ...
    FOR i IN 1..100 LOOP
        tabnbr(i) := i+1;
    END LOOP
END;
```



Extraction de données

- **Particularité** : utilisation de la clause INTO
- **Syntaxe** : `SELECT liste INTO nomVariablePLSQL FROM Nomtable;`
- **Règles** :
 - clause INTO : obligatoire
 - le SELECT doit obligatoirement ramener une seule ligne et une seule : sinon
 - Une requête qui renvoie plusieurs enregistrements, génère une erreur PL/SQL en déclenchant l'exception ORA-01422 : **TOO_MANY_ROWS**
 - Une requête qui renvoie aucun enregistrement, génère une erreur PL/SQL en déclenchant des exceptions ORA-01403: **NO_DATA_FOUND**



Conflit de noms

- Si une variable porte le même nom qu'une colonne d'une table, c'est la colonne qui l'emporte

Exemple :

```
DECLARE
```

```
    name varchar(30) := 'toto';
```

```
BEGIN
```

```
    DELETE FROM emp where name = name;
```

```
...
```

  ici DELETE entraine la suppression de tous les employés et nom seulement l'employé 'toto'

- Pour éviter ça, le plus simple est de ne pas donner de nom de colonne à une variable

3

Structures de contrôle



Présentation générale

○ Modifier le déroulement logique des instructions en utilisant des structures de contrôle

○ Structures de contrôle conditionnel (Instruction IF)

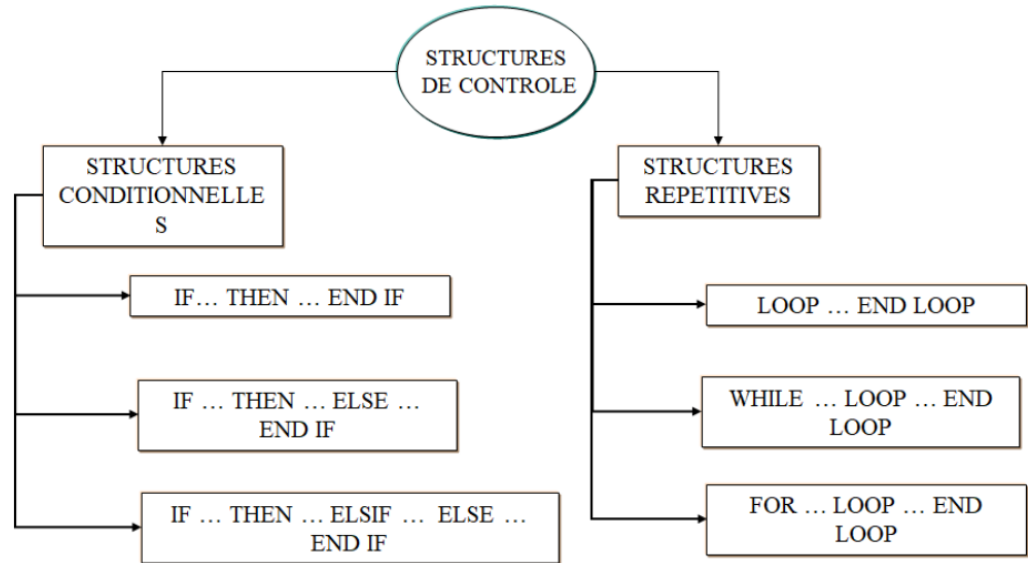
○ Structures de Contrôle Itératif

○ Boucle de base LOOP

○ Boucle FOR

○ Boucle WHILE

○ – Instruction EXIT





Structure conditionnelle if

- Avec les structures conditionnelles IF et CASE, on peut déclencher des actions en fonction du résultat des conditions
- Remarques
 - ELSIF en un seul mot
 - END IF en deux mots
 - Une seule clause ELSE est permise

```
IF condition  
THEN  
instructions;  
END IF;
```

```
IF condition THEN  
instructions1;  
ELSE  
instructions2;  
END IF;
```

```
IF condition1 THEN  
instructions1;  
ELSIF condition2 THEN  
instructions2;  
ELSIF ...  
...  
ELSE  
instructionsN;  
END IF;
```



Exemples

```
IF salaire <=1000 THEN
    nouveau_salaire := ancien_salaire + 100;
ELSEIF salaire > 1000 AND emp_id_emp = 1 THEN
    nouveau_salaire := ancien_salaire + 500;
ELSE
    nouveau_salaire := ancien_salaire + 300;
END IF;
```

```
DECLARE
    vmention char(2);
    vnote NUMBER(4,2);
BEGIN
...
IF vnote >=16 THEN
    vmention:='TB';
ELSIF vnote>=14 AND vnote<16 THEN
    vmention='B';
ELSIF vnote >=12 AND vnote<14 THEN
    vmention:='AB';
ELSIF vnote>=10 AND vnote<12 THEN
    vmention:='P';
ELSE vmention:='R';
END IF;
...
```




Structure **choix CASE**

Syntaxe :

```
CASE expression
  WHEN expr1 THEN instruction1;
  WHEN expr2 THEN instruction2;
  ...
  ELSE instructionN;
END CASE;
```

Ou :

```
CASE
  WHEN condition1 THEN
    instructions1;
  WHEN condition2 THEN
    instructions2;
  ...
  ELSE instructionsN;
END CASE;
```

Exemple :

```
DECLARE
  vmention char(2);
  vnote NUMBER(4,2);
BEGIN
  ...
  CASE
    WHEN vnote >=16 THEN
      vmention:='TB';
    WHEN vnote >=14 THEN
      vmention='B';
    WHEN vnote >=12 THEN
      vmention:='AB';
    WHEN vnote >=10 THEN
      vmention:='P';
    ELSE vmention:='R';
  END CASE;
  ...
```



Syntaxe :

```
LOOP
  instructions;
  EXIT WHEN
    condition
END LOOP;
```

- Obligation d'utiliser la commande EXIT

Exemple :

```
DECLARE
  Vsomme NUMBER(4):=0;
  Ventier NUMBER(3):=1;
BEGIN
  LOOP
    vsomme:=vsomme+ventier;
    ventier:=ventier+1;
    EXIT WHEN ventier>100
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(('SOMME='|| vsomme);
END;
/
```



Syntaxe :

```
WHILE condition LOOP
    instructions;
END LOOP;
```

Exemple :

```
Vsomme NUMBER(4):=0;
Ventier NUMBER(3):=1;
BEGIN
    WHILE ventier<=100 LOOP
        vsomme:=vsomme+ventier;
        ventier:=ventier+1;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(('SOMME='||
        vsomme));
END;
/
```



Syntaxe :

```
FOR compteur IN inf..sup LOOP  
  instructions;  
END LOOP;
```

Exemple :

```
DECLARE  
vsomme NUMBER(4):=0;  
BEGIN  
  FOR ventier in 1..100 LOOP  
    vsomme := vsomme + ventier;  
  END LOOP;  
  DBMS_OUTPUT.PUT_LINE('SOMME='|| vsomme );  
  
END;  
/
```



Exercice

- Écrivez un bloc PL/SQL qui affiche la somme des nombres entre 1000 et 10000.

```
SET SERVEROUTPUT ON;

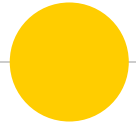
DECLARE

    somme NUMBER := 0;

BEGIN

    FOR i IN 1000..10000 LOOP
        somme := somme + i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Somme = ' || somme);
END;

/
```



Fin du chapitre 2